

CS301: Operating Systems & Architecture

Lecture 8: Virtual Memory, Paging, and Page Replacement Algorithms

Instructor: Dr. Alan Sterling | Department of Computer Science & Engineering | Semester Fall 2026

1. Introduction to Virtual Memory

Virtual memory is a memory management capability of an operating system that uses hardware and software to allow a computer to compensate for physical memory shortages by temporarily transferring data from random access memory (RAM) to disk storage. It provides each process with its own contiguous, private logical address space, entirely decoupled from physical memory hardware constraints.

Key Benefits of Virtual Memory:

- Multiprogramming beyond physical RAM capacity: Processes larger than physical RAM can execute seamlessly.
- Memory Protection: Isolation prevents errant or malicious processes from reading or overwriting other processes' memory spaces.
- Shared Memory: Libraries (such as libc or DirectX) can be shared across multiple processes via read-only shared physical frames.
- Efficient Process Creation: System calls like fork() utilize Copy-On-Write (COW) semantics.

2. Paging Mechanism & Address Translation

In a paging system, the logical address space is divided into fixed-size blocks called pages (typically 4 KB in size). Physical memory is partitioned into matching fixed-size blocks called frames. Address translation is orchestrated in hardware by the Memory Management Unit (MMU).

Logical Address Structure:

A virtual address consists of two components: [Page Number (p) | Offset (d)].

- Page Number (p): Index into the process page table to identify the base physical frame number (f).
- Offset (d): Displacement within the page/frame. The offset is copied directly into the physical address.

Mathematical decomposition: For page size $S = 2^n$, Offset $d = \text{Address} \bmod S$, Page Number $p = \text{Address} / S$.

3. Translation Lookaside Buffer (TLB) and Effective Access Time

Accessing a page table in physical memory adds memory bus latency to every instruction. To mitigate this overhead, CPUs employ a Translation Lookaside Buffer (TLB), a fast associative hardware cache storing recent page-to-frame translations.

Effective Access Time (EAT) Formula:

Let α be the TLB hit ratio, ϵ be the TLB search time (~ 1 ns), and m be the main memory access time (~ 100 ns).

Formula: $EAT = \alpha * (\epsilon + m) + (1 - \alpha) * (\epsilon + 2m)$

When accounting for Page Fault Rate p (where disk access time is ~ 10 ms = 10,000,000 ns):

$EAT = (1 - p) * m + p * (\text{Page Fault Overhead Time})$

Even if $p = 0.001$ (one fault in 1000 references), the effective memory access latency degrades by a factor of nearly 100x.

4. Page Replacement Algorithms

When a page fault occurs and all physical memory frames are currently allocated, the operating system must choose a victim frame to evict. If the victim frame is marked dirty (modified), it must be written to disk; if clean, it can simply be overwritten.

1. First-In, First-Out (FIFO):

Replaces the oldest loaded page in memory. Suffers from Belady's Anomaly, where increasing the number of physical frames allocated to a process results in MORE page faults rather than fewer.

2. Optimal Page Replacement (OPT / MIN):

Replaces the page that will not be used for the longest period of time in the future. Has the lowest possible page fault rate of any algorithm. However, OPT is impossible to implement in practice because future reference strings cannot be known by the OS. It serves exclusively as a benchmark.

3. Least Recently Used (LRU):

Replaces the page that has not been referenced for the longest period in the past. LRU is a stack algorithm, which mathematically guarantees that it NEVER suffers from Belady's Anomaly. Implemented using hardware reference timestamps or doubly linked lists.

5. Thrashing and the Working Set Model

Thrashing is a catastrophic operating system condition where processes spend more time paging in and out of secondary storage than executing CPU instructions. As page fault rates skyrocket, CPU utilization plunges toward zero. The OS misinterprets low CPU utilization as a sign that multiprogramming should be increased, launching additional processes and worsening the thrashing cycle.

Peter Denning's Working Set Strategy:

The Working Set Model defines WSS_i as the set of pages referenced by process i within the most recent delta time window. The operating system monitors the sum of all working set sizes. If $\text{sum}(WSS_i) > \text{Total Available Memory Frames}$, the system is thrashing, and the OS must temporarily suspend one or more processes, swap them to disk completely, and reallocate their frames to the remaining active processes.

Exam Takeaways & Common Pitfalls:

- Remember: Belady's Anomaly impacts FIFO; LRU and OPT are provably exempt.
- Dirty Bit: Always check whether eviction requires a disk write.
- Paging eliminates external fragmentation, but causes internal fragmentation.